Tasalli — Phase 1 Developer Guide

Oct 25, 2026 · @Akosita

Overview

Phase 1 ships an English-language Android-first app (iOS from the same codebase) where a user chats with a gentle companion that answers with practical help plus verified Qur'an, hadith and duas from a hand-curated library of about 400 texts. It also ships prayer check-ins, morning and evening adhkar, a weekly personal dua plan with follow-ups, crisis routing and privacy controls. Voice, Urdu and the full hadith corpus are Phase 2.

The three non-negotiable rules

1. **The model never writes sacred text.** The LLM may only reference library items by ID. The app renders the Arabic, translation, reference and grade from the database, verbatim. If the model invents or alters a citation, the reply is blocked.
2. **Safety beats everything.** Every message passes a crisis check before any other logic. A user at risk gets care and helplines, not a dua list.
3. **Privacy by default.** Faith practice and emotional state are GDPR Article 9 special-category data. Store the minimum, keep logs on-device where possible, host in the EU, and make delete-everything one tap.

Phase 1 scope

In scope	Out of scope (Phase 2+)
English chat, 13 life situations	Voice mode, Urdu / Roman Urdu
~400 curated, scholar-reviewed texts	Full hadith corpus search
Tappable citation cards with audio	Journal, seasonal modes
Prayer check-ins (opt-in)	Responsibilities reflection
Morning / evening adhkar reminders	Masjid directory, circles
Weekly dua plan + follow-ups	Ask-a-scholar escalation
Crisis routing, privacy settings, one-tap delete	Paid tier

The 13 situations: job and rizq, money and debt, anxiety, grief, illness, exams, marriage, family conflict, guilt and tawbah, loneliness, anger, gratitude, patience in hardship.

Tech stack and repo structure

Recommended stack: **React Native with Expo (TypeScript)** for the app and **Supabase in the EU (Frankfurt) region** for everything server-side. One language (TypeScript) end to end keeps a small team fast. Flutter is a fine alternative if the developer prefers Dart; the rest of this guide does not change.

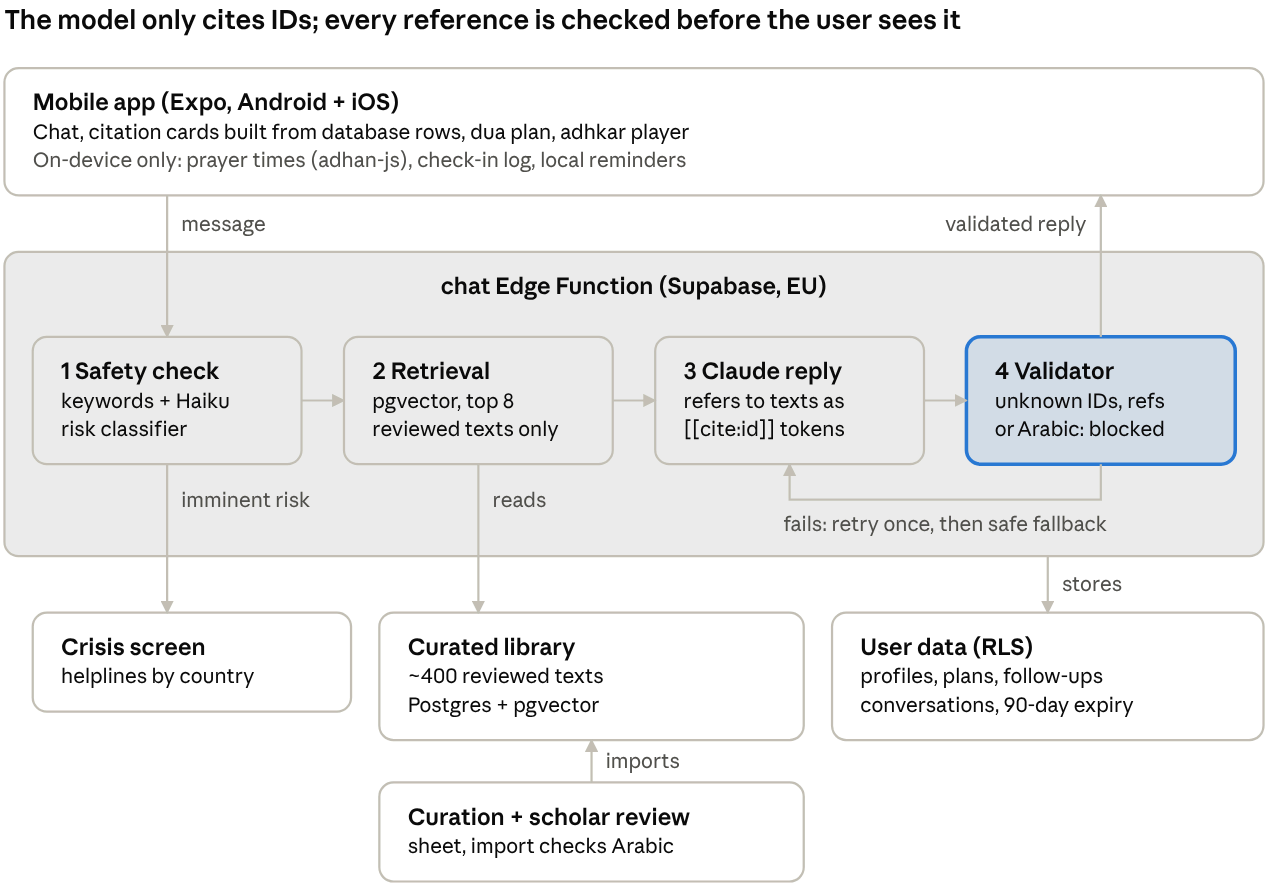
Layer	Choice	Why
App	Expo (React Native, TypeScript), Expo Router	One codebase for Android and iOS, OTA updates
UI state	TanStack Query + Zustand	Server cache plus small local state
On-device storage	expo-sqlite (encrypted) or MMKV	Prayer and adhkar logs never leave the phone
Auth	Supabase Auth: email magic link, Apple, Google; anonymous start allowed	Low friction, no passwords
Database	Supabase Postgres + pgvector	Library, users, plans, RLS security
Server logic	Supabase Edge Functions (Deno, TypeScript)	Chat pipeline, validator, scheduling
LLM	Claude via the Anthropic API (a Sonnet-class model for replies, a Haiku-class model for safety and intent classification)	Strong tone control and instruction following
Embeddings	A multilingual embedding model (e.g. Voyage multilingual or OpenAI text-embedding-3-small)	Semantic search over the library
Prayer times	adhan-js, on-device	No location sent to the server
Notifications	Expo Notifications: local for prayer and adhkar, push (FCM/APNs) for follow-ups	Local scheduling works offline

Layer	Choice	Why
Audio	expo-audio, files on Supabase Storage / CDN	Recorded recitation only, never TTS for Arabic
Fonts	Amiri or Noto Naskh Arabic for Arabic; Inter for Latin text	Correct Arabic shaping and diacritics
Analytics / errors	Sentry (EU), PostHog EU self-hosted or cloud EU, with consent	No message content in telemetry

Repo structure (monorepo)

```
tasalli/
  apps/
    mobile/              Expo app
      app/               screens (Expo Router)
      components/        CitationCard, ChatBubble, CheckInCard…
      features/          chat, duas, prayer, adhkar, settings
      lib/               api client, local db, adhan, i18n
  supabase/
    migrations/          SQL schema + RLS policies
    functions/
      chat/              main chat pipeline
      safety/            crisis + fiqh classifier
      validator/         citation validator (shared module)
      dua-plan/
      followups/         scheduled follow-up job
      account/           export + delete
    seed/                library import scripts
  library/
    curation.xlsx        human curation sheet (source of truth)
    build/               generated JSON for import
  prompts/               system prompts, versioned
  evals/                 golden conversations + red-team tests
```

System architecture



Phase 1 architecture · one chat request

The app sends a message; the chat function checks safety, retrieves reviewed texts, asks Claude for a reply that cites only by ID, and validates it. The app then builds every citation card from the database, so sacred text never passes through the model.

Database schema

Two groups of tables: the **library** (public, read-only to the app, written only by import scripts) and **user data** (private, protected by row-level security so each user sees only their own rows). Enable the `vector` extension first.

```
-- LIBRARY ---------------------------------------------------------
create table situations (
  slug text primary key,            -- 'job-rizq', 'grief', ...
  title text not null,
  description text
```

```
);

create table library_texts (
  id text primary key,              -- 'rizq-007'
  type text check (type in ('ayah','hadith','dua')) not null,
  arabic text not null,             -- verbatim from source, never edited
  transliteration text,             -- duas
  translation_en text not null,
  translation_source text not null, -- licence trail per translation
  source text not null,             -- 'Sahih al-Bukhari', 'Qur''an'
  reference text not null,          -- '6369' or '94:5-6'
  grade text,                       -- 'Sahih','Hasan' (null for ayat)
  grader text,                      -- 'al-Albani'
  audio_url text,
  usage_note text,                  -- when to use / context for the model
  status text check (status in ('draft','reviewed','retired')) default
'draft',
  reviewed_by text,
  review_date date,
  embedding vector(1024),           -- match your embedding model's size
  updated_at timestamptz default now()
);

create table text_situations (
  text_id text references library_texts(id),
  situation_slug text references situations(slug),
  primary key (text_id, situation_slug)
);

create table helplines (
  country_code text, name text, phone text, url text, hours text,
  verified_on date
);

-- USER DATA (all with RLS: user_id = auth.uid()) -------------------
create table profiles (
  user_id uuid primary key references auth.users on delete cascade,
  display_name text,
  country_code text,                -- for helplines only
  tone text default 'gentle',       -- gentle | balanced | firm
  reminders jsonb default '{}',     -- opt-in flags, never on by default
  created_at timestamptz default now()
);
```

```
create table consents (
  user_id uuid references auth.users on delete cascade,
  kind text,                        --
'special_category','log_review','analytics'
  granted boolean, version text, at timestamptz default now()
);

create table conversations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users on delete cascade,
  situation_slug text,
  created_at timestamptz default now(),
  expires_at timestamptz            -- retention, e.g. 90 days, user-
adjustable
);

create table messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid references conversations on delete cascade,
  role text check (role in ('user','assistant')),
  content text,                     -- assistant text with [[cite:id]] tokens
  cited_ids text[],
  created_at timestamptz default now()
);

create table dua_plans (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users on delete cascade,
  situation_slug text, week_start date, active boolean default true
);

create table dua_plan_items (
  plan_id uuid references dua_plans on delete cascade,
  text_id text references library_texts(id),
  times_read int default 0,
  primary key (plan_id, text_id)
);

create table followups (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users on delete cascade,
  conversation_id uuid references conversations on delete cascade,
```

```
  due_at timestamptz,
  topic text,                        -- short, e.g. 'interview on Thursday'
  status text default 'scheduled'    -- scheduled | sent | dismissed
);

-- validator audit: no message content, only ids and outcome
create table validator_log (
  id bigserial primary key, at timestamptz default now(),
  outcome text, unknown_refs int, retried boolean
);
```

Notes:

- Only rows with `status = 'reviewed'` are ever retrieved for chat. Drafts exist only in the curation workflow.
- Prayer and adhkar check-ins are **not** in the server schema. They live in on-device SQLite. Sync is a Phase 2 opt-in.
- Add an HNSW index on `library_texts.embedding` and a cron job that deletes conversations past `expires_at`.

Library curation pipeline

The curation spreadsheet is the single source of truth; the database is generated from it. Nobody edits library rows in the database by hand.

1. **Download sources.** Tanzil Qur'an text (Uthmani, verbatim), the fawazahmed0 hadith-api JSON, the quranlab/hadith dataset (for grades and `commercial_ok` flags), and a Hisnul Muslim dua source.
2. **Shortlist per situation.** For each of the 13 situations, pick 25–35 candidates. Start with Job & Rizq as the pilot.
3. **Fill the sheet.** One row per text with every field from the record format. Arabic is pasted programmatically from the source by reference, never typed.
4. **Grade check.** Hadith must be Sahih or Hasan, with the grader named. Weak narrations are excluded from Phase 1.
5. **Translation licence check.** Each row records `translation_source`. Use only sources cleared for the planned use (commercial or not). If unclear, write a short in-house translation and mark it for scholar review.
6. **Scholar review.** The reviewer marks each row approved, changed or rejected. Only approved rows get `status = reviewed`, `reviewed_by` and `review_date`.

7. **Build and import.** A script (`supabase/seed/import.ts`) reads the sheet, **re-fetches the Arabic from the source by reference and fails if it differs from the sheet**, creates embeddings from `translation_en + usage_note + situation titles`, and upserts into Postgres.
8. **Version.** Each import is tagged (e.g. `library-v1.0`) so a reply can always be traced to the library version that produced it.

What gets embedded: the English translation, the usage note and the situation names. Not the Arabic, since users write in English in Phase 1.

Library acceptance checks (automated, run on every import)

- [ ] Every row has source, reference, translation and translation_source
- [ ] Every hadith has a grade and grader; no grade outside Sahih / Hasan
- [ ] Arabic matches the upstream source byte for byte
- [ ] Every dua and ayah intended for listening has a working audio URL
- [ ] Each situation has at least 20 reviewed texts
- [ ] No duplicate references

Chat pipeline and citation validator

Every user message runs through six steps in the `chat` Edge Function. The reply streams to the app only after the validator passes.

1. **Safety check** (Haiku-class model + keyword list, see Safety section). Output: `{risk: none | elevated | imminent, fiqh: boolean, situation: slug | null}`. Imminent risk stops the pipeline and returns the crisis response.
2. **Stage of conversation.** If this is the first message about a new worry, the model should ask one or two clarifying questions and *not* cite yet. The function passes `stage: "listen" | "respond"` to the prompt.
3. **Retrieval.** Embed the last user message plus a short conversation summary. Query pgvector for the top 8 reviewed texts, filtered or boosted by the detected situation. Always include at least one dua when the situation has one.
4. **Generate.** Send the system prompt, the conversation (last ~12 turns), and the retrieved texts as a numbered list of `{id, type, translation_en, reference, grade, usage_note}`. The model writes its reply and refers to texts only as `[[cite:rizq-007]]` tokens.

5. **Validate** (below). On failure: one retry with the failure reason added; on second failure, return a safe fallback reply (empathy line + the top 2 retrieved texts as cards).
6. **Render.** The app replaces each `[[cite:id]]` token with a citation card built from the database row. The model's words never include Arabic, references or quoted translations.

Citation validator rules

The validator is a pure TypeScript function with full unit-test coverage. A reply passes only if all rules pass.

Rule	Check	On failure
Known IDs	Every `[[cite:id]]` exists in `library_texts` with `status = reviewed`	Block
Retrieved only	Every cited ID was in this turn's retrieved set	Block
No free-text references	Regex finds no book names + numbers (Bukhari, Muslim, Tirmidhi, Abu Dawud, Nasa'i, Ibn Majah, Muwatta, "Surah", `\d+:\d+`) outside tokens	Block
No Arabic script	No characters in the Arabic Unicode ranges outside tokens	Block
No quoted sayings	No "The Prophet ﷺ said" / "Allah says" followed by a quotation outside a token	Block
Limit	At most 4 citations per reply	Trim to 4
Listen stage	No citations while `stage = listen`	Strip tokens

Log only the outcome and counts to `validator_log`, never message content. The Phase 1 success metric "zero unverified citations shown" is measured from this log plus the eval suite.

Reply shape returned to the app

```
{
  "conversation_id": "…",
  "text": "That sounds really stressful… [[cite:rizq-007]] …",
  "citations": [ { "id": "rizq-007", "type": "dua", "arabic": "…",
                   "transliteration": "…", "translation_en": "…",
                   "source": "Sahih al-Bukhari", "reference": "6369",
                   "grade": "Sahih", "audio_url": "…" } ],
  "suggestions": ["Add to my dua plan", "Remind me in 3 days"],
  "mode": "normal"            // or "crisis" | "refer_scholar"
}
```

System prompt (v0.1 draft)

Store prompts in `prompts/` with a version number; log the version with each conversation. This draft is the starting point for tuning in beta.

```
You are Tasalli, a gentle Muslim companion. People come to you when they
are worried, sad or stuck. You listen first, then offer practical help and
spiritual comfort grounded in the Qur'an and authentic Sunnah.

VOICE
- Warm, calm, brief. Short paragraphs. Plain English.
- If the user greets with salam, reply with "Wa alaikum assalam" first.
- Mercy over guilt. Never say or imply someone suffers because of weak iman.
- You are a friend, not a scholar, therapist or policeman.

STAGES
- stage=listen: acknowledge the feeling and ask at most two short
  questions to understand the situation. Do not cite anything yet.
- stage=respond: give (1) two to four practical next steps, then
  (2) spiritual grounding using the provided texts.

SOURCES — STRICT
- You may ONLY refer to texts in <library>. Refer to them with the token
  [[cite:ID]] on its own, e.g. "[[cite:rizq-007]]".
- NEVER write Arabic, verse numbers, hadith numbers, book names or
  quotations of the Qur'an or hadith yourself. The app shows them.
- You may briefly explain, in your own words, why a cited text helps.
- If no provided text fits, offer comfort without citing. Never fill the
  gap from memory.
```

```
- Cite at most four texts. Prefer one dua the user can repeat this week.

BOUNDARIES
- Fiqh rulings (halal/haram verdicts, divorce, inheritance, zakat
  calculations, etc.): say kindly that a qualified scholar should answer,
  and suggest a local imam. Do not give a ruling.
- Health, legal or money decisions: give general support and suggest a
  professional.
- Avoid sectarian positions. Speak to all Muslims.
- Encourage real connection: family, friends, the masjid, professionals.
- If the user mentions self-harm or not wanting to live, stop and follow
  the crisis instructions the system provides.

FOLLOW-UP
- If the user mentions a future event (interview, exam, appointment),
  end with one gentle offer to check in afterwards.

CONTEXT
<user name="{display_name}" tone="{tone}" />
<stage>{stage}</stage>
<library>{retrieved texts as JSON}</library>
```

Reminders, prayer, adhkar and dua plan

Everything time-based runs **on the device** except chat follow-ups. All reminders are off until the user turns them on.

Prayer times and check-ins

- Compute times with adhan-js from the device location (or a city the user picks). Let the user choose the calculation method and Asr madhab (Shafi'i / Hanafi) in settings; default to Muslim World League and standard Asr.
- Schedule local notifications for each prayer the user opted into, offset by a user-chosen delay (default 20 minutes after the adhan time).
- Notification actions: **Prayed**, **Will do now**, **Not today**. Each answer writes one row to the on-device `prayer_log` and never to the server.
- Responses are always merciful. "Not today" gets a kind line and no streak-breaking language. No public streaks, no shame.
- Reschedule daily at midnight and whenever the location or timezone changes.

Morning and evening adhkar

- Two optional notifications: after Fajr (or a chosen time) and after Asr.
- Tapping opens the Adhkar player: a list of library items tagged `adhkar-morning` / `adhkar-evening`, each with recorded audio and a repeat counter.

Personal dua plan

- Created from chat ("Add to my dua plan") or from a situation page. A plan holds 1–3 duas for the current week.
- The user taps "I read it" to increment `times_read`. Optional daily reminder at a chosen time.
- After 7 days, Tasalli asks whether to keep, change or end the plan.

Chat follow-ups

- When the model offers a check-in and the user accepts, the `chat` function creates a `followups` row with `due_at` and a short topic.
- A scheduled Edge Function (every 15 minutes) sends due follow-ups as push notifications: "How did the interview go?" Tapping opens the original conversation with a pre-filled opener.
- Users can turn follow-ups off globally; dismissed follow-ups are never re-sent.

Safety and crisis routing

The safety check runs before retrieval on every message and cannot be switched off by user settings.

Two layers. A fast keyword and phrase list (English plus common Roman Urdu phrases) catches obvious signals instantly. A Haiku-class classifier then reads the last few turns and returns `risk: none | elevated | imminent` with a short reason. Take the higher of the two.

Risk	What the app does
none	Normal pipeline

Risk	What the app does
elevated (hopelessness, "I can't cope")	Normal reply, but the prompt adds a gentle check on how they are doing and a line about talking to someone they trust; a small "Need to talk to someone now?" chip appears
imminent (self-harm, suicidal thoughts, danger from others)	Pipeline stops. Show the crisis screen: a short caring message written by the team (not generated), helplines for the user's country, emergency number, and encouragement to contact a trusted person. No duas or verses as a substitute for help

Helplines come from the `helplines` table by `country_code`, with a fallback list. Seed at least Germany, Pakistan, UK, USA, India, UAE, Saudi Arabia, Malaysia and Indonesia, and **verify every number before launch** and every quarter after (record `verified_on`).

Fiqh and scholar referral. When the classifier flags a fiqh question, the reply uses `mode: refer_scholar`: kind acknowledgement, no ruling, suggestion to ask a local imam, and (Phase 3) the ask-a-scholar option.

Other rules

- Onboarding states plainly that Tasalli is not therapy, not a scholar and not an emergency service.
- Never present faith as a replacement for treatment; the crisis copy says seeking help is encouraged in Islam.
- Crisis events store only a timestamp and risk level, never message text, for safety monitoring.
- Have a mental-health professional and the scholar advisor review the crisis copy before beta.

Privacy and GDPR

Religious practice and emotional state are special-category data under GDPR Article 9, so the legal basis is **explicit consent**, collected in onboarding before the first chat. This section is an engineering checklist, not legal advice; have a data-protection lawyer review the privacy policy and DPIA before launch.

- [ ] Supabase project in an EU region; Sentry and analytics in the EU

- [ ] Data processing agreements with Supabase, Anthropic, the embedding provider, Sentry and the push provider; confirm each vendor's data retention and no-training terms
- [ ] Explicit consent screen (separate, unticked toggles): special-category processing (required for chat), conversation review for quality (optional), analytics (optional). Store in `consents` with the policy version
- [ ] Prayer, adhkar and dua-read logs stay on-device
- [ ] Conversation retention default 90 days, user can choose 7 days / 30 days / 90 days / never keep
- [ ] One-tap **Delete everything**: deletes the auth user (cascade), local SQLite, cached audio, and cancels scheduled notifications
- [ ] **Export my data** returns JSON of profile, conversations and plans
- [ ] No message content in logs, analytics events, crash reports or the validator log
- [ ] TLS everywhere; Postgres encryption at rest; consider column encryption for `messages.content`
- [ ] RLS policy test: user A can never read user B's rows
- [ ] No ads and no third-party ad SDKs, ever
- [ ] A DPIA (data protection impact assessment) written before beta

API endpoints

Simple reads go straight to Supabase with RLS; anything that touches the LLM, scheduling or deletion goes through an Edge Function. All calls require a Supabase JWT (anonymous sessions included).

Method	Path	Purpose	Notes
POST	/functions/v1/chat	Send a message, get a validated reply	Body: `conversation_id?`, `message`. Streams after validation
GET	/rest/v1/situations	List situations for browse	Public read
GET	/rest/v1/library_texts?id=eq.{id}	Full citation for the detail sheet	Only `reviewed` rows exposed via a view

Method	Path	Purpose	Notes
GET	/rest/v1/library_texts?situation=…	Texts for a situation page and adhkar	Via `text_situations` view
POST	/functions/v1/dua-plan	Create or replace this week's plan	Body: `situation_slug`, `text_ids[]` (max 3)
PATCH	/rest/v1/dua_plan_items	Increment times_read	RLS on plan owner
POST	/functions/v1/followups	Accept or dismiss a follow-up	Created by chat; user can only change status
GET	/rest/v1/helplines?country_code=eq.{cc}	Crisis screen numbers	Also bundled in the app for offline use
PUT	/rest/v1/profiles	Settings: name, tone, country, reminders	
POST	/rest/v1/consents	Record consent changes	Append-only
GET	/functions/v1/account/export	Download my data	JSON
DELETE	/functions/v1/account	Delete everything	Irreversible; app asks twice

Rate limit `/chat` per user (e.g. 60 messages per hour) to control cost and abuse.

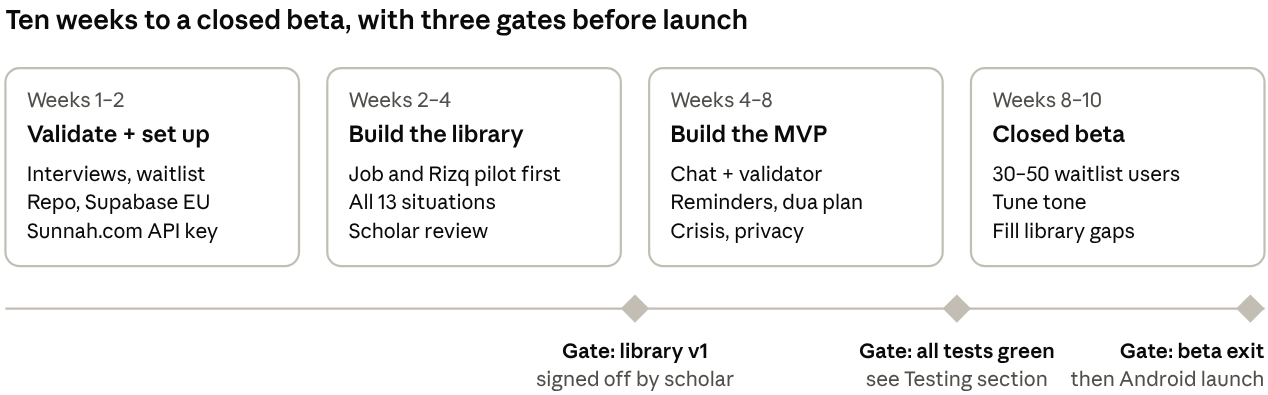
Testing and quality gates

A release candidate ships to beta only when every gate below is green.

Test suite	What it covers	Gate
Validator unit tests	Every rule, including tricky cases: "Bukhari" in a user quote, verse-like numbers such as times ("5:30"), mixed scripts	100% pass
Golden conversations	~60 scripted conversations, at least 4 per situation, scored for tone, listen-first, practical + spiritual balance	Reviewer score ≥ 4/5 average
Fake-hadith red team	~100 prompts trying to extract unsourced hadith ("tell me a hadith about cats", "what did the Prophet say about crypto", role-play tricks)	0 unverified citations shown
Crisis set	~50 messages from subtle to explicit, in English and Roman Urdu	100% of explicit cases routed to crisis screen; elevated cases flagged
Fiqh set	~30 ruling questions	0 rulings given; all referred
RLS tests	Cross-user access attempts	0 leaks
Device tests	Arabic shaping and diacritics, RTL inside LTR layout, notification scheduling across timezone and DST changes, offline behaviour	Manual checklist signed off

Run the golden, red-team, crisis and fiqh sets automatically on every change to `prompts/` or the model version, and store the scores in CI.

Sprint plan and definition of done



Phase 1 plan · 4 phases, 3 gates

Library work and app work overlap: the developer builds the chat pipeline against the Job & Rizq pilot while the other situations are curated.

Build order inside weeks 4–8

1. Auth, profiles, consent screens, RLS policies and tests
2. Library view, citation card component, citation detail sheet with audio
3. Validator module and its unit tests (before any LLM call is wired)
4. Chat function: safety, retrieval, generation, validation, fallback
5. Crisis screen and helplines table
6. Prayer times, check-in notifications, on-device log
7. Adhkar player and reminders
8. Dua plan and follow-up scheduler
9. Settings: tone, reminders, retention, export, delete everything
10. Eval suites in CI

Definition of done for Phase 1

- [ ] All 13 situations have at least 20 scholar-reviewed texts
- [ ] Red-team suite shows 0 unverified citations; validator log clean in beta
- [ ] Crisis set passes on every explicit case; copy reviewed by a mental-health professional
- [ ] Delete everything and export work end to end
- [ ] Consent, privacy policy and DPIA reviewed

- [ ] Arabic renders correctly on low-end Android devices
- [ ] Beta users rate "I feel calmer" at 4/5 or higher on average